

Competitive Programming

Section 54 Editorial: Geometry

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Orientation	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Segment Intersection	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Polygon Double Area	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Orientation. For points a, b, c , compute the cross product $(b-a) \times (c-a)$. Positive means c is to the left of directed line $a \rightarrow b$, negative means right, and zero means collinear. This sign is the signed doubled area of triangle abc , so it is robust with integer coordinates.

B. Segment Intersection. Two segments intersect properly when the endpoints of each segment lie on opposite sides of the other segment. Collinear cases are handled by checking whether an endpoint lies inside the bounding box of the other segment. Combining orientation signs with `on_segment` covers crossing, touching endpoints, and overlapping collinear segments.

C. Polygon Area. The shoelace formula sums $x_i y_{i+1} - y_i x_{i+1}$ around the polygon. This is the sum of signed triangle areas against the origin. The absolute value divided by two is the area; many contest problems print doubled area to avoid fractions.

Common pitfalls. Use 64-bit integers for cross products. Include the edge from the last vertex back to the first. For intersections, collinear cases are not optional.

All three geometry tasks are solved with integer cross products.

- A computes the sign of `cross(AB, AC)` to classify orientation.
- B combines orientation tests with bounding-box checks for collinear endpoint and overlap cases.
- C uses the shoelace sum and prints twice the polygon area.

No floating point is needed. The main implementation risk is overflow, so C++ solutions use `long long` for coordinate products.

2 A. Orientation

2.1 Restatement

Classify point C relative to directed segment AB.

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int T;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
```

```
    ax,ay,bx,by,cx,cy=data[idx:idx+6]; idx+=6; v=(bx-ax)*(cy-ay)-(by-ay)*(cx-ax); out.append(v)
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Segment Intersection

3.1 Restatement

Decide whether two closed segments intersect.

3.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

3.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

3.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using P=pair<long long,long long>; long long ori(P a,P b,P c){return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])}
```

3.6 Full Python solution

```
import sys
def ori(a,b,c): return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
def between(a,b,c): return min(a[0],b[0])<=c[0]<=max(a[0],b[0]) and min(a[1],b[1])<=c[1]<=max(a[1],b[1])
def inter(a,b,c,d):
    o1,o2,o3,o4=ori(a,b,c),ori(a,b,d),ori(c,d,a),ori(c,d,b)
    if o1==0 and between(a,b,c): return True
```

```

    if o2==0 and between(a,b,d): return True
    if o3==0 and between(c,d,a): return True
    if o4==0 and between(c,d,b): return True
    return (o1>0)!=(o2>0) and (o3>0)!=(o4>0)
def main():
    data=list(map(int,sys.stdin.buffer.read().split())); t=data[0]; idx=1; out=[]
    for _ in range(t):
        x=data[idx:idx+8]; idx+=8; out.append("YES" if inter((x[0],x[1]),(x[2],x[3]),(x[4],x[5]),(x[6],x[7])) else "NO")
    print("\n".join(out))
if __name__=="__main__": main()

```

4 C. Polygon Double Area

4.1 Restatement

Print twice the area of a polygon in vertex order.

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. `\square`

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; pts=[(data[i],data[i+1]) for i in range(1,2*n+1,2)]; s=0
    for i in range(n):
```

```
        x1,y1=pts[i]; x2,y2=pts[(i+1)%n]; s+=x1*y2-y1*x2
    print(abs(s))
if __name__=="__main__": main()
```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.